

Cryptography 101

What is Cryptography?

Key Goals:

- **Confidentiality:** Ensuring information is accessible only to authorized parties
- **Authentication:** Verifying the identity of parties
- **Integrity:** Guaranteeing data hasn't been altered
- **Non-repudiation:** Preventing denial of actions taken

Basic Concepts

Plaintext

- Original readable message
- Before encryption

Ciphertext

- Encrypted message
- Unreadable without key

Key

- Secret information used in encryption/decryption
- Different types for different systems

Algorithm

- Mathematical procedure for encryption/decryption

Hashing

One-Way Functions

What is Hashing?

A hash function takes an input and produces a **fixed-size** output (hash/digest).

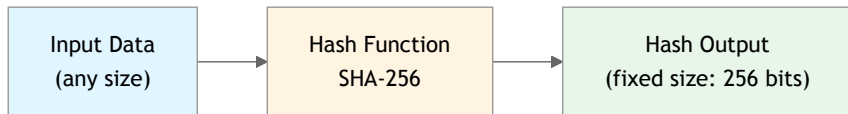
Properties:

-  **Deterministic:** Same input always produces same output
-  **Fast to compute:** Efficient calculation
-  **One-way:** Cannot reverse the hash to get original input
-  **Avalanche effect:** Small input change = completely different hash
-  **Collision resistant:** Hard to find two inputs with same hash

Common Uses:

- Password storage
- Data integrity verification
- Digital signatures
- Blockchain

Hashing Process



Examples:

Input: "Hello, World!"

SHA-256: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e

Input: "Hello, World" (removed !)

SHA-256: dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f

Input: "password123"

SHA-256: ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

Input: "The quick brown fox jumps over the lazy dog"

SHA-256: d7a8fbb307d7809469ca9abcb0082e4f8d5651e46d3cdb762d02d0bf37c9e592

Input: "" (empty string)

SHA-256: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

Popular Hash Functions

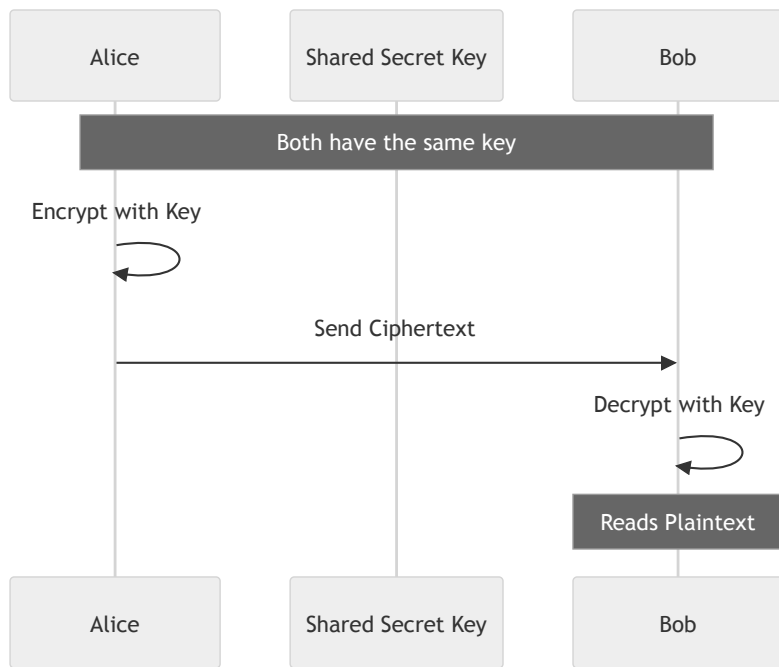
Algorithm	Output Size	Status	Use Cases
MD5	128 bits	⚠ Broken	Legacy systems (avoid)
SHA-1	160 bits	⚠ Deprecated	Git commits (transitioning)
SHA-256	256 bits	✅ Secure	Bitcoin, certificates, general use
SHA-3	Variable	✅ Secure	Modern applications
bcrypt	184 bits	✅ Secure	Password hashing
Argon2	Variable	✅ Secure	Password hashing (latest)

Symmetric Cryptography

Same Key for Encryption & Decryption

Symmetric Encryption

Both parties use the **same secret key** to encrypt and decrypt messages.



Symmetric Encryption

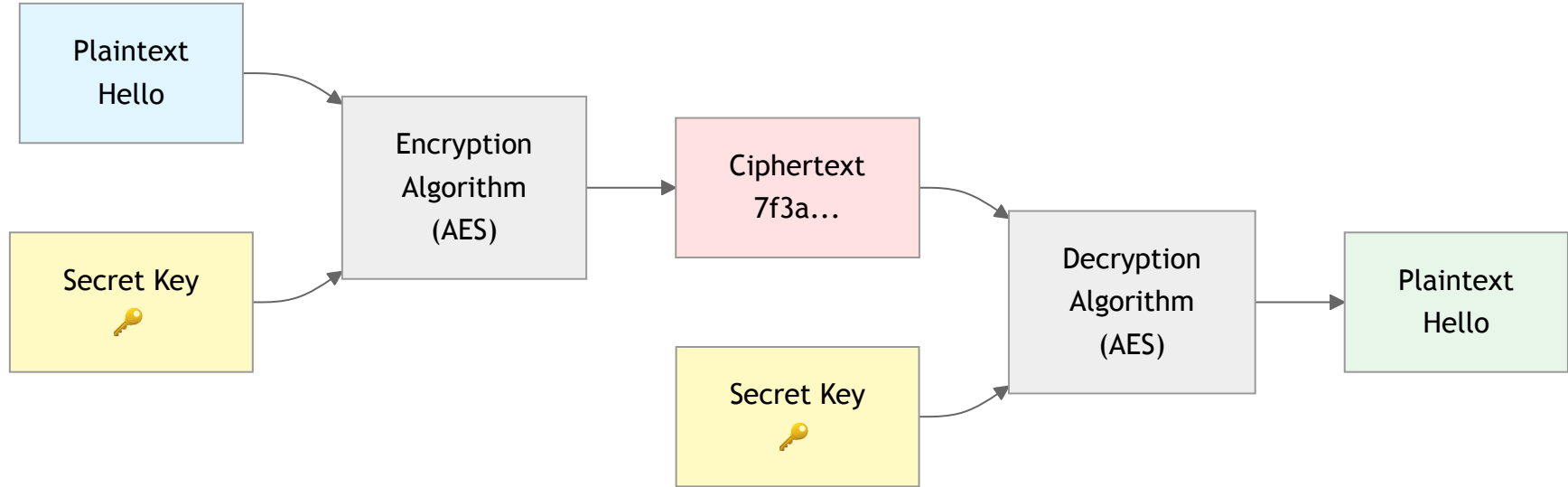
Advantages:

- ⚡ Fast and efficient
- 💪 Strong security with proper key length

Challenges:

- 🔑 Key distribution problem
- 🗃️ Many keys needed ($n \times (n-1)/2$ for n users)

Symmetric Encryption Process



Common Symmetric Algorithms

Algorithm	Key Size	Block Size	Status
DES	56 bits	64 bits	✗ Insecure
3DES	168 bits	64 bits	⚠️ Deprecated
AES-128	128 bits	128 bits	✓ Secure
AES-256	256 bits	128 bits	✓ Highly Secure
ChaCha20	256 bits	Stream	✓ Modern

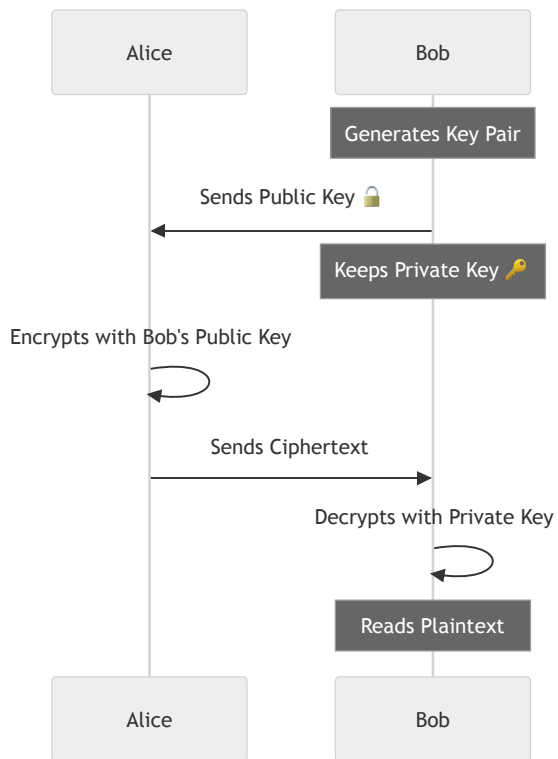
AES (Advanced Encryption Standard) is the most widely used today

Asymmetric Cryptography

Public Key Cryptography

Asymmetric Encryption

Uses a **pair of keys**: public key (shareable) and private key (secret).

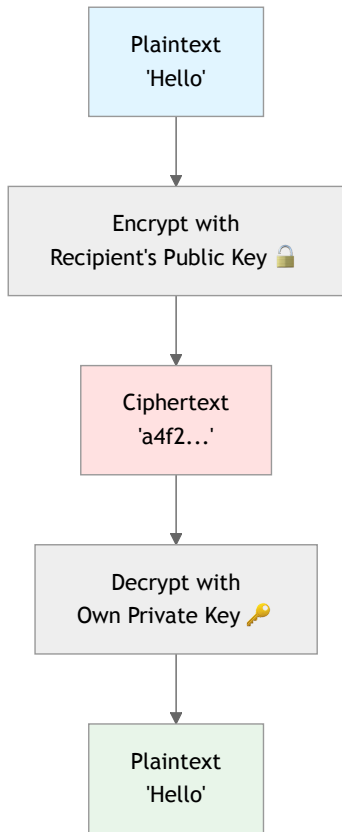


Asymmetric Encryption

Key Properties:

-  Public key can be shared freely
-  Private key must remain secret
-  What's encrypted with public key can only be decrypted with private key

Asymmetric Encryption Process



Asymmetric vs Symmetric

Feature	Symmetric	Asymmetric
Keys	One shared key	Key pair (public + private)
Speed	⚡ Very fast	🐢 Slower (100-1000x)
Key Distribution	⚠️ Difficult	✅ Easy (public key)
Key Management	Many keys for many users	One key pair per user
Use Cases	Bulk data encryption	Key exchange, signatures

Hybrid Approach (SSL/TLS):

1. Use asymmetric crypto to exchange a symmetric key
2. Use symmetric crypto for actual data encryption
3. Get best of both worlds! 🎯

Common Asymmetric Algorithms

Algorithm	Key Size	Based On	Use Cases
RSA	2048-4096 bits	Factorization	Encryption, signatures
DSA	2048-3072 bits	Discrete log	Digital signatures
ECC	256-521 bits	Elliptic curves	Modern systems
Diffie-Hellman	2048+ bits	Discrete log	Key exchange
Ed25519	256 bits	Edwards curve	SSH keys, signing

Note: ECC provides equivalent security with much smaller keys (256-bit ECC $\approx$ 3072-bit RSA)

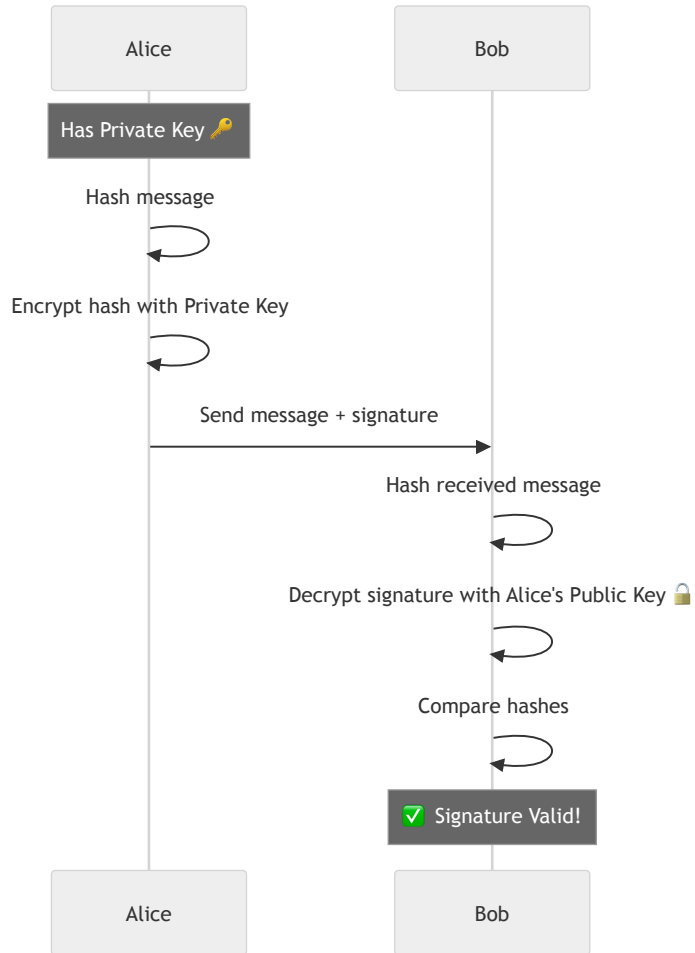
Digital Signatures

Authentication & Integrity

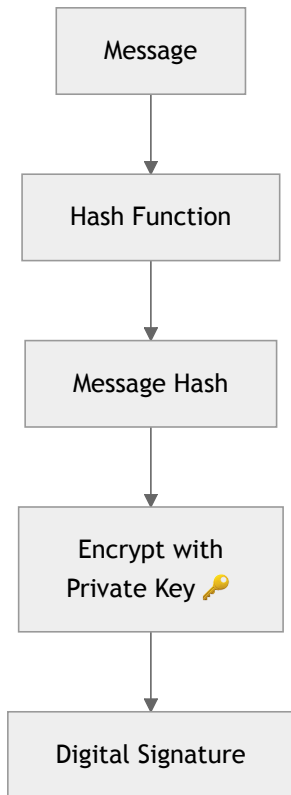
What are Digital Signatures?

Digital signatures prove:

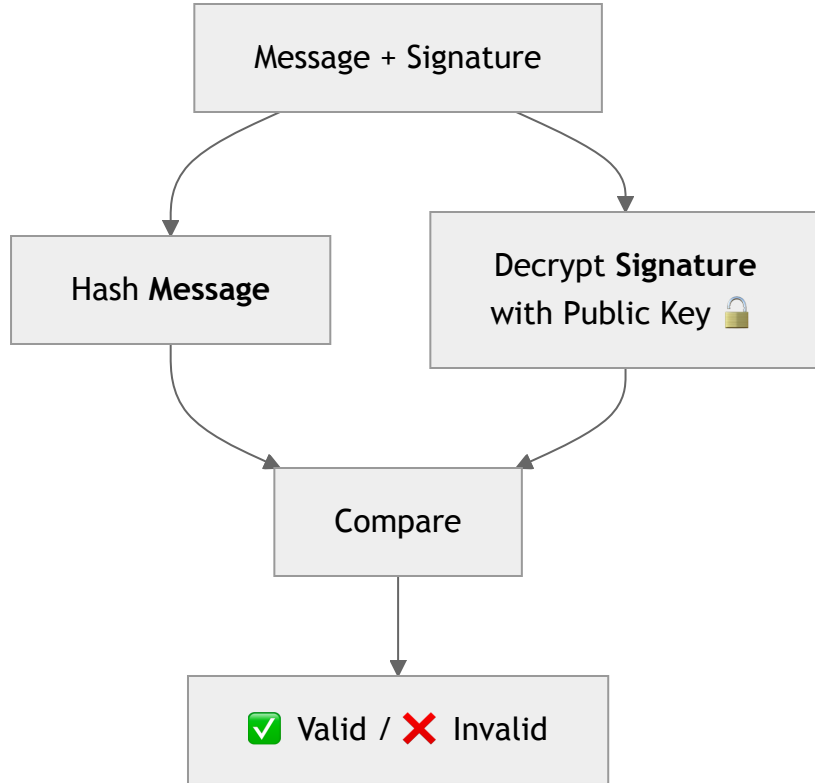
- 🖋️ **Authentication:** Who created the message
- 🔒 **Integrity:** Message hasn't been modified
- 🚫 **Non-repudiation:** Sender can't deny sending it



Signature Creation

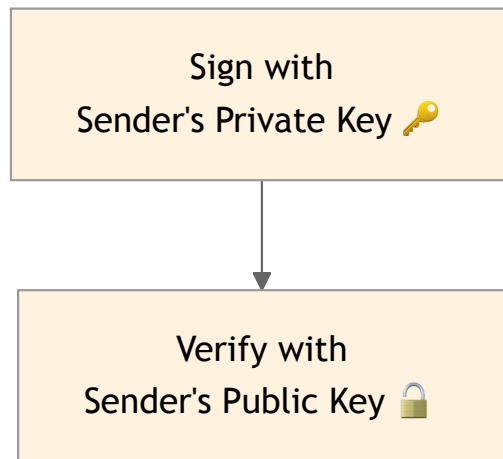
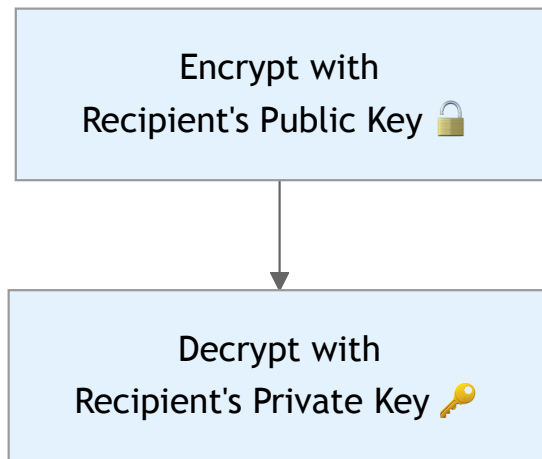


Signature Verification



Signature vs Encryption

Common Confusion:



Certificates & PKI

Public Key Infrastructure

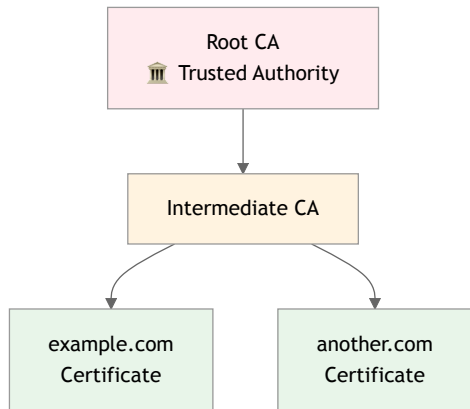
Digital Certificates

A certificate binds a public key to an identity.

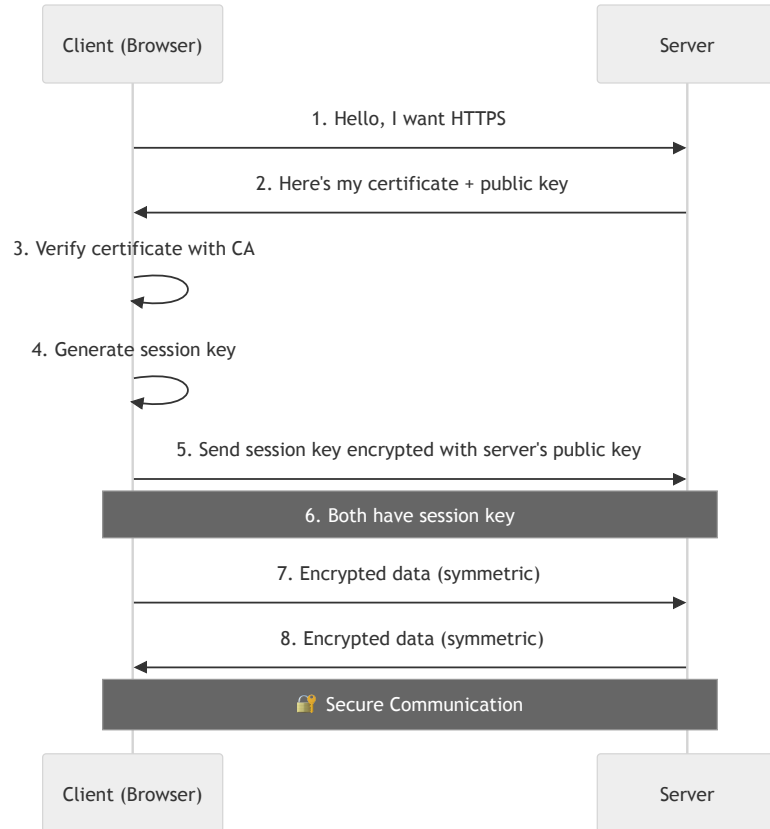
Certificate Contains:

- Subject's name (domain, organization)
- Subject's public key
- Issuer (Certificate Authority)
- Validity period
- Digital signature from CA

Digital Certificates



How HTTPS Works



Best Practices

Staying Secure

Cryptography Best Practices

DO:

-  Use well-established algorithms (AES, RSA, SHA-256)
-  Use adequate key lengths (RSA ≥ 2048 , AES ≥ 128)
-  Use trusted libraries (OpenSSL, libsodium)
-  Keep software updated
-  Use authenticated encryption (AES-GCM)
-  Generate truly random keys

Cryptography Best Practices

DON'T:

- ❌ Roll your own crypto
- ❌ Use deprecated algorithms (MD5, DES, RC4)
- ❌ Reuse keys across different purposes
- ❌ Store keys in code or version control
- ❌ Use weak passwords
- ❌ Ignore warnings about certificates

Summary

Key Takeaways

Concept	Purpose	Example
Hashing	Integrity, one-way	SHA-256 for passwords
Symmetric	Fast bulk encryption	AES for file encryption
Asymmetric	Key exchange, authentication	RSA for TLS handshake
Signatures	Verification, non-repudiation	Sign software releases
Certificates	Identity verification	HTTPS websites

Thank You!
Questions?